Wrapper Classes in Java :-

*) In java, wrapper class allows primitive data types to be represented as Objects.

*) This allows primitives to be used in Object-Oriented features such as collections, generics and APIs that require objects.

*) Each wrapper class encapsulates a corresponding primitive values inside an object.

| Primitive | | Wrapper class | Examples :- |
|---|---|---|---|
| byte | ⇒ | Byte | byte b = 1; |
| short | ⇒ | Short | Byte byteObj = Byte.valueOf(b); |
| int | ⇒ | Integer | int i = 10; |
| long | ⇒ | Long | Integer intObj = Integer.valueOf(i); |
| float | ⇒ | Float | float f = 18.6f; |
| double | ⇒ | Double | Float floatObj = Float.valueOf(f); |
| char | ⇒ | Character | char c = 'a'; |
| boolean | ⇒ | Boolean | Character charObj = c; // autoboxing. |

double d = 250.5;

Double doubObj = Double.valueOf(d);

Why wrapper class needed?

1) Java Collections (ArrayList, HashMap, etc) stores only objects, not primitives.

2) Wrapper Objects allows primitives to be used in

Object-oriented features like methods, synchronization, and serialization.

3) Objects supports null values, while primitives do not.

4) Wapper class provides utility methods such as: compareTo(), equals() and toString().

Autoboxing :-

Conversion of Primitive data type → objects

Eg :- int → Integer, long → Long, etc....

```
char ch = 'a';
Character c = ch; // Autoboxing : char -> character
ArrayList<Integer> list = new ArrayList<>();
list.add(25); // Autoboxing : int -> Integer.
```

Unboxing :-

Conversion of a wrapper class → primitives.

Eg :- Integer → int, Long → Long, etc....

```
Character ch = 'a';
char c = ch; // Unboxing: Character -> char
ArrayList<Integer> list = new ArrayList<>();
list.add(84);
int n = list.get(0); // unboxing: Integer -> int
```

Methods of Wrapper Class :-

1. parseXxx(String s)
   → converts a String into given primitive type
   → int a = Integer.parseInt("100");
   sout(a); // 100 → primitive

2. valueOf(String s)
   → converts string into wrapper object
   → Integer a = Integer.valueOf("100");
   sout(a); // 100 → object

3. valueOf(primitive)
   → converts primitive value into wrapper object
   → Integer a = Integer.valueOf(10);
   sout(a); // 10

4. xxxValue()
   → Converts wrapper objects into its primitive type.
   → Integer obj = 50;
   int a = obj.intValue();
   sout(a); // 50

5. toString ()

-> Converts wrapper object into a String

-> String s = Integer.toString (10);

sout (s); // "10"

6. compareTo (Xxx obj)

-> compares two wrapper objects

-> Integer a = 10, b = 20;

sout (a.compareTo (b)); // -1

meaning:

- a < b -> -1
- a == b -> 0
- a > b -> 1

7. equals (Object obj)

-> Compares values not refrences

-> Integer a = 100, b = 100;

sout (a.equals (b)); // true

8. hashCode ()

-> Return hash code of the object

-> Integer a = 100;

sout (a.hashCode ()); // 100

for Integer,
hashcode = value,
for string, or other
hashcode = random value.
String s = "Hello"
hashcode = 69609650

9. min(x, y), max(x, y), sum(x, y)

→ Returns minimum, maximum, sum of two values

→ ~~In~~ Sout (Integer.min(10, 20)); //10

sout (Integer.max(10, 20)); //20

sout (Integer.sum(10, 20)); //30

10. compare(x, y)

→ Compares two primitive values

→ sout (Integer.compare(10, 20)); // -1

meaning :-

- Same logic as "compareTo()"
- compare() → primitive, compareTo() → wrapper

11. isNaN()

→ checks if value is Not a Number (Float/Double only)

→ double val = 0.0/0.0;

sout (Double.isNaN(val)); // true

12. isInfinite()

→ Checks infinite value (Float / Double only)

→ double val = 10.0/0.0;

sout (Double.isInfinite(val)); // true

13. decode (String s)

-> Decodes decimal, hex or octal string

-> sout (Integer.decode ("0XA")); // 10

sout (Integer.decode ("010")); // 8

sout (Integer.decode ("10")); // 10

Explanation:

- 0XA -> Hex -> 10
- 010 -> Octal -> 8
- 10 -> Decimal -> 10

14. parseBoolean (String s)

-> converts string to boolean

-> sout (Boolean.parseBoolean ("true")); // true

sout (Boolean.parseBoolean ("false")); // false

sout (Boolean.parseBoolean ("Hello")); // false

meaning :-

- Only "true" -> true
- Everything else -> false

Things To know :-

```
Integer a = 100;
Integer b = 100;
sout (a == b);
```

O/P :-

True

```
Integer a = 200;
Integer b = 200;
sout (a == b)
```

O/P :-

False

Why?

Due to Integer cache (-128 to 127)

- Java has a cache range : -128 to 127
- within this range :
  1. Same object is reused
  2. No new object is created
- Outside this range :
  1. New object created everytime

```
Integer a = 200; // new object
Integer b = 200; // another new object.
```

"==" -> compares refrences (memory address)

a == b -> false

```
Integer a = 100; // new object
Integer b = 100; // same object (no another new object)
```

Rules :-

1. Use "==" for primitive (int, double, etc...)
2. Use ".equals()" for Objects.

Because it compares values, not references.

Do you know !

```
Integer a = 200;
Integer b = 200;
int c = 200;
sout (a == b); //false
sout (a == c); //true
```

```
Integer a = 200;
int c = 200;
a == c;
```

O/P :- True

How ?

1. Types are different :

a → Integer (object), c → int (primitive)

So, java can't directly compare object with primitive.

2. Auto-unboxing happens :

java automatically converts : a → a.intValue()

So, expression becomes : a.intValue() == c  200 == 200

3. Now both are primitives,

200 == 200 | value comparison if it is primitive not object.

So, Result : true

```
Integer a = null;
int b = 100;
sout (a == b); ×
```

× Runtime Error : NullPointerException

why ?

- Java tries "a.intValue()"
- But "a = null"